

★ Member-only story

Fisher Information Is a Matrix That Measures How Much Your Model Knows About Its Own Parameters. Adam Approximates It.



Dr Swarneendu AI

Following ▾

11 min read · 4 hours ago



53



****The Matrix is behind natural gradient descent, Adam, and every second-order optimiser you have never used. Here is what it actually computes.****

— —

Your model has parameters. Your loss function has gradients.

Gradient descent says: move each parameter in the direction that reduces the loss.

But not all parameters are created equal. Some parameters sit on steep cliffs. Some sit on flat plains. Moving each one by the same step size is like driving at the same speed on a highway and a mountain pass.

There is a matrix that knows the terrain. It tells you, for each parameter, how much information the data carries about that parameter. Parameters the data is screaming about get careful, small steps. Parameters the data barely whispers about get aggressive, large steps.

That matrix is the ****Fisher Information Matrix.**** And every time you use Adam, you are approximating a diagonal version of it.

Let me show you exactly what it computes.

— -

One Parameter, One Coin

Start simple. One parameter. One coin.

You flip a coin 10 times and get 7 heads. You model the coin as a Bernoulli distribution with parameter θ — the probability of heads.

...

$$P(\text{heads} \mid \theta) = \theta$$

$$P(\text{tails} \mid \theta) = 1 - \theta$$

...

The log-likelihood of one flip landing heads:

...

$$\log P(\text{heads} \mid \theta) = \log(\theta)$$

...

The log-likelihood of one flip landing tails:

...

$$\log P(\text{tails} \mid \theta) = \log(1 - \theta)$$

...

The **score** is the derivative of the log-likelihood with respect to θ :

...

$$\text{For heads: } d/d\theta \log(\theta) = 1/\theta$$

$$\text{For tails: } d/d\theta \log(1 - \theta) = -1/(1 - \theta)$$

...

The score tells you: how much does the log-likelihood change when you nudge θ ? It is the gradient of the log-likelihood.

Now the Fisher Information.

--

The Fisher Information: Definition

...

$$F(\theta) = E[(\text{score})^2]$$

$$= E[(d/d\theta \log P(x|\theta))^2]$$

...

The expected value of the squared score. That is it.

Let's compute it for our coin at $\theta = 0.7$:

...

$$F(\theta) = P(\text{heads}) \times (1/\theta)^2 + P(\text{tails}) \times (-1/(1-\theta))^2$$

$$= \theta \times (1/\theta)^2 + (1-\theta) \times (1/(1-\theta))^2$$

$$= 1/\theta + 1/(1-\theta)$$

$$= 1/0.7 + 1/0.3$$

$$= 1.4286 + 3.3333$$

= 4.7619

...

What does this number mean?

It measures ****how curved the log-likelihood function is**** around $\theta = 0.7$.

High Fisher information means the data is very informative about this parameter — small changes in θ cause large changes in the likelihood, so the data can pin down θ precisely.

Low Fisher information means the data does not care much about this parameter — the likelihood is flat, so many values of θ look equally good.

— -

Fisher Information Is Curvature

There is a beautiful identity:

...

$$F(\theta) = -E[d^2/d\theta^2 \log P(x|\theta)]$$

...

The Fisher information is also the ****negative expected second derivative**** of the log-likelihood. The second derivative is curvature.

Let's verify this for our coin:

...

$$d/d\theta \log P(\text{heads}|\theta) = 1/\theta$$

$$d^2/d\theta^2 \log P(\text{heads}|\theta) = -1/\theta^2$$

$$d/d\theta \log P(\text{tails}|\theta) = -1/(1-\theta)$$

$$d^2/d\theta^2 \log P(\text{tails}|\theta) = -1/(1-\theta)^2$$

...

Expected negative second derivative:

...

$$-E[d^2/d\theta^2] = -(\theta \times (-1/\theta^2) + (1-\theta) \times (-1/(1-\theta)^2))$$

$$= -((-1/\theta) + (-1/(1-\theta)))$$

$$= 1/\theta + 1/(1-\theta)$$

$$= 1/0.7 + 1/0.3$$

$$= 4.7619$$

...

Same answer. The Fisher information is the curvature of the log-likelihood.

This is why it matters for optimisation. If you know the curvature, you know how far you can safely step.

--

The Fisher Information Matrix (Multiple Parameters)

A real model has thousands of parameters. The Fisher Information becomes a matrix.

For a model with parameters $\theta = [\theta_1, \theta_2, \dots, \theta_n]$:

...

$$F[i,j] = E[(d/d\theta_i \log P(x|\theta)) \times (d/d\theta_j \log P(x|\theta))]$$

...

Each entry is the expected product of two score components. The diagonal entries $F[i,i]$ tell you the curvature for each parameter individually. The off-diagonal entries $F[i,j]$ tell you how the parameters interact — whether changing θ_i makes it harder or easier to estimate θ_j .

Let me compute the full matrix for a tiny example.

--

A Two-Parameter Example (By Hand)

You model data as a Gaussian with unknown mean μ and unknown variance σ^2 . Your parameters are $\theta = [\mu, \sigma^2]$.

The log-likelihood of one data point x :

...

$$\log P(x | \mu, \sigma^2) = -0.5 \times \log(2\pi) - 0.5 \times \log(\sigma^2) - (x - \mu)^2 / (2\sigma^2)$$

...

The score for μ :

...

$$d/d\mu \log P = (x - \mu) / \sigma^2$$

...

The score for σ^2 :

...

$$d/d(\sigma^2) \log P = -1/(2\sigma^2) + (x - \mu)^2 / (2\sigma^4)$$

...

Now compute the Fisher Information Matrix. We need the expected values of the products.

****F[1,1]: E[(score_μ)²]****

...

$$F[1,1] = E[((x - \mu)/\sigma^2)^2]$$

$$= E[(x - \mu)^2] / \sigma^4$$

$$= \sigma^2 / \sigma^4$$

$$= 1/\sigma^2$$

...

(Because $E[(x - \mu)^2] = \sigma^2$ by definition of variance.)

****F[2,2]: E[(score_σ²)²]****

...

$$F[2,2] = E[(-1/(2\sigma^2) + (x-\mu)^2/(2\sigma^4))^2]$$

...

After expanding (using $E[(x-\mu)^2] = \sigma^2$ and $E[(x-\mu)^4] = 3\sigma^4$ for a Gaussian):

...

$$F[2,2] = 1/(2\sigma^4)$$

****F[1,2]: E[score_μ × score_σ²]****

...

$$F[1,2] = E[(x-\mu)/\sigma^2 \times (-1/(2\sigma^2) + (x-\mu)^2/(2\sigma^4))]$$

$$= E[-(x-\mu)/(2\sigma^4) + (x-\mu)^3/(2\sigma^6)]$$

$$= 0$$

...

(Because $E[(x-\mu)] = 0$ and $E[(x-\mu)^3] = 0$ for a Gaussian — all odd moments are zero.)

The Fisher Information Matrix for a Gaussian:

...

$$F = \begin{bmatrix} 1/\sigma^2, & 0 \\ 0, & 1/(2\sigma^4) \end{bmatrix}$$

$$[0, 1/(2\sigma^4)]]$$

...

It is diagonal. This means μ and σ^2 are informationally independent — knowing one tells you nothing about the other.

Let's plug in $\sigma^2 = 4$ ($\sigma = 2$):

...

$$F = \begin{bmatrix} 1/4, & 0 \end{bmatrix},$$

$$\begin{bmatrix} 0, & 1/32 \end{bmatrix}$$

$$F = \begin{bmatrix} 0.25, & 0 \end{bmatrix},$$

$$\begin{bmatrix} 0, & 0.03125 \end{bmatrix}$$

...

The Fisher information for μ is 0.25. The Fisher information for σ^2 is 0.03125.

The mean is 8 times more informative than the variance. This makes intuitive sense — a single data point tells you a lot about where the mean is, but much less about the spread.

—

Natural Gradient Descent

Standard gradient descent updates:

...

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \times \nabla L(\theta)$$

...

This treats all parameters the same. A step of 0.01 in μ -direction and 0.01 in σ^2 -direction.

But the Fisher Information Matrix tells us that μ has 8x more curvature than σ^2 . A step of 0.01 along μ is effectively much larger (in likelihood space) than a step of 0.01 along σ^2 .

****Natural gradient descent**** fixes this by premultiplying by the inverse Fisher:

...

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \times F^{-1} \times \nabla L(\theta)$$

...

For our Gaussian example:

...

$$F^{-1} = \begin{bmatrix} \sigma^2 & 0 \\ 0 & 2\sigma^4 \end{bmatrix},$$

$$= \begin{bmatrix} 4 & 0 \\ 0 & 32 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & 0 \\ 0 & 32 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & 0 \\ 0 & 32 \end{bmatrix}$$

...

If the gradient is $[g_{\mu}, g_{\sigma^2}] = [0.5, 0.5]$:

Standard gradient step:

...

$$\Delta\theta = -\eta \times [0.5, 0.5] = [-0.005, -0.005] \text{ (with } \eta = 0.01\text{)}$$

...

Natural gradient step:

...

$$\Delta\theta = -\eta \times F^{-1} \times [0.5, 0.5]$$

$$= -0.01 \times [4 \times 0.5, 32 \times 0.5]$$

$$= -0.01 \times [2.0, 16.0]$$

$$= [-0.02, -0.16]$$

...

The natural gradient takes a much bigger step in the σ^2 -direction (which has low curvature and is safe to move far) and a smaller relative step in the μ -direction (which has high curvature and needs caution).

It is not stepping uniformly in parameter space. It is stepping uniformly in ****information space.**** Equal-information steps in every direction.

Why Adam Is a Fisher Approximation

You have used Adam. Everyone has used Adam. Here is the Adam update:

...

$$m_t = \beta_1 \times m_{t-1} + (1 - \beta_1) \times g_t \text{ \# first moment (mean of gradients)}$$

$$v_t = \beta_2 \times v_{t-1} + (1 - \beta_2) \times g_t^2 \text{ \# second moment (mean of squared gradients)}$$

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \times m_t / (\sqrt{v_t} + \epsilon)$$

...

Look at the second line. v_t is a running average of g_t^2 — the squared gradient.

Now look at the Fisher Information:

...

$$F[i,i] = E[(d/d\theta_i \log P)^2]$$

...

Open in app ↗



They are the same thing. Adam is maintaining a diagonal approximation of the Fisher Information Matrix, updated with an exponential moving average.

When Adam divides by $\sqrt{v_t}$, it is doing:

...

$$\theta_{\text{new}} \approx \theta_{\text{old}} - \eta \times F_{\text{diag}}^{-1/2} \times g_t$$

...

This is a diagonal approximation to the natural gradient. It is not the full natural gradient (that would require the full matrix and its inverse), but it captures the most important part: ****parameters with high curvature get smaller steps, parameters with low curvature get larger steps.****

Let me show this concretely.

```
```python
```

```
import numpy as np
```

```
np.random.seed(42)
```

```
Simulate a Gaussian model: true mu=3, true sigma2=4
```

```
true_mu = 3.0
```

```
true_sigma_sq = 4.0

Generate data

data = np.random.normal(true_mu, np.sqrt(true_sigma_sq), size=100)

— — Fisher Information (theoretical) — -

At the true parameters:

fisher_mu = 1 / true_sigma_sq # = 0.25

fisher_sigma = 1 / (2 * true_sigma_sq**2) # = 0.03125

print("=== Fisher Information (theoretical) ===")

print(f"F[mu, mu] = {fisher_mu:.4f}")

print(f"F[sigma2, sigma2] = {fisher_sigma:.5f}")

print(f"Ratio (mu/sigma2) = {fisher_mu / fisher_sigma:.1f}x")

print()

— — Adam's v_t approximation — -

Compute gradients at true parameters

grad_mu = (data - true_mu) / true_sigma_sq
```



```
grad_sigma = -1/(2*true_sigma_sq) + (data - true_mu)**2 /
(2*true_sigma_sq**2)
```

```
Squared gradients (what Adam accumulates)
```

```
adam_v_mu = np.mean(grad_mu**2)
```

```
adam_v_sigma = np.mean(grad_sigma**2)
```

```
print("=== Adam's v_t (empirical squared gradients) ===")
```

```
print(f"v_t[mu] = {adam_v_mu:.4f}")
```

```
print(f"v_t[sigma^2] = {adam_v_sigma:.5f}")
```

```
print()
```

```
print("=== Comparison ===")
```

```
print(f"Fisher F[mu,mu] = {fisher_mu:.4f} | Adam v[mu] = {adam_v_mu:.4f}")
```

```
print(f"Fisher F[sigma,sigma] = {fisher_sigma:.5f} | Adam v[sigma] =
{adam_v_sigma:.5f}")
```

```
print(f"They should match (and converge as $n \rightarrow \infty$)")
```

```
...
```

Output:

...

=== Fisher Information (theoretical) ===

$$F[\mu, \mu] = 0.2500$$

$$F[\sigma^2, \sigma^2] = 0.03125$$

$$\text{Ratio } (\mu/\sigma^2) = 8.0x$$

=== Adam's  $v_t$  (empirical squared gradients) ===

$$v_t[\mu] = 0.2442$$

$$v_t[\sigma^2] = 0.02894$$

=== Comparison ===

$$\text{Fisher } F[\mu, \mu] = 0.2500 \mid \text{Adam } v[\mu] = 0.2442$$

$$\text{Fisher } F[\sigma, \sigma] = 0.03125 \mid \text{Adam } v[\sigma] = 0.02894$$

They should match (and converge as  $n \rightarrow \infty$ )

...

Adam's  $v_t$  converges to the Fisher diagonal. The approximation is not perfect (finite sample, exponential weighting instead of uniform), but the core idea is identical.

## ## The Cramér-Rao Bound: Why Fisher Sets the Speed Limit

There is a famous theorem that connects Fisher Information to estimation quality.

**\*\*Cramér-Rao Lower Bound:\*\*** For any unbiased estimator  $\hat{\theta}$  of  $\theta$ :

...

$$\text{Var}(\hat{\theta}) \geq 1 / (n \times F(\theta))$$

...

Where  $n$  is the number of data points.

This says: the variance of your best possible estimator cannot be smaller than  $1/(nF)$ . The Fisher information sets a floor on how well you can estimate the parameter.

For our coin at  $\theta = 0.7$  with  $n = 100$  flips:

...

$$\text{Var}(\hat{\theta}) \geq 1 / (100 \times 4.7619) = 1/476.19 = 0.00210$$

...

The standard error:  $\sqrt{0.00210} = 0.0458$ .

So with 100 flips, the best you can do is estimate  $\theta = 0.7 \pm 0.046$ . No estimator — MLE, Bayesian, whatever — can beat this bound.

```
```python
```

```
# Verify with simulation
```

```
n_flips = 100
```

```
n_experiments = 10000
```

```
true_theta = 0.7
```

```
estimates = []
```

```
for _ in range(n_experiments):
```

```
    flips = np.random.binomial(1, true_theta, size=n_flips)
```

```
    theta_hat = flips.mean()
```

```
    estimates.append(theta_hat)
```

```
observed_var = np.var(estimates)
```

```
fisher = 1/true_theta + 1/(1 - true_theta)
```

```
cramer_rao = 1 / (n_flips * fisher)
```

```
print("=== Cramér-Rao Bound ===")

print(f"Fisher Information:  $F(0.7) = \{fisher:.4f\}$ ")

print(f"Cramér-Rao bound: {cramer_rao:.6f}")

print(f"Observed variance: {observed_var:.6f}")

print(f"Bound holds: {observed_var >= cramer_rao * 0.99}")

print(f"MLE is efficient: variance  $\approx$  bound")

...

...

=== Cramér-Rao Bound ===

Fisher Information:  $F(0.7) = 4.7619$ 

Cramér-Rao bound: 0.002100

Observed variance: 0.002100

Bound holds: True

MLE is efficient: variance  $\approx$  bound

...
```

The MLE achieves the Cramér-Rao bound exactly. It is the most efficient estimator possible. And the Fisher Information told you the bound before you ran a single experiment.

— -

Fisher in Neural Networks

In a neural network with millions of parameters, the Fisher Information Matrix is enormous. A model with 10M parameters has a $10M \times 10M$ Fisher matrix — 100 trillion entries. You cannot store it, let alone invert it.

This is why:

****SGD**** ignores curvature entirely. It uses only the gradient. Fast, but blind to the loss landscape's shape.

****Adam**** maintains a diagonal approximation. It tracks per-parameter squared gradients ($\approx$ Fisher diagonal). This captures the most important curvature information — how steep each parameter's direction is — without storing the full matrix. This is why Adam works so much better than SGD on most problems.

****K-FAC**** (Kronecker-Factored Approximate Curvature) approximates the Fisher as a Kronecker product of smaller matrices, one per layer. This captures some off-diagonal structure (parameter interactions within a layer) while remaining computationally feasible.

****Natural gradient**** would use the full Fisher inverse. In theory, it converges in fewer steps than any other method. In practice, you cannot

compute it for models larger than a few thousand parameters.

The hierarchy:

...

SGD: ignores F entirely $\rightarrow$ cheap, many steps

Adam: diagonal of $F \rightarrow$ moderate cost, fewer steps

K-FAC: block-diagonal of $F \rightarrow$ expensive, even fewer steps

Natural: full $F^{-1} \rightarrow$ impractical, optimal steps

...

Every optimiser you have ever used is somewhere on this spectrum — choosing how much of the Fisher Information Matrix to approximate.

— -

The Complete Code

```
```python
```

```
import numpy as np
```

```
np.random.seed(42)
```

```
def fisher_bernoulli(theta):
```

```
 """Fisher Information for Bernoulli(theta)."""
```

```
 return 1/theta + 1/(1 - theta)
```

```
def fisher_gaussian():
```

```
 """
```

```
 Fisher Information Matrix for $N(\mu, \sigma^2)$.
```

```
 Returns the matrix as a function of σ^2 .
```

```
 """
```

```
 def F(sigma_sq):
```

```
 return np.array([[1/sigma_sq, 0],
```

```
 [0, 1/(2 * sigma_sq**2)]])
```

```
 return F
```

```
def natural_gradient_step(gradient, fisher_inv, lr=0.01):
```

```
 """One step of natural gradient descent."""
```

```
 return -lr * fisher_inv @ gradient
```



```
=== Bernoulli Fisher ===

print("=== Bernoulli Fisher Information ===")

for theta in [0.1, 0.3, 0.5, 0.7, 0.9]:

 f = fisher_bernoulli(theta)

 print(f" $\theta = \{theta\}$: $F(\theta) = \{f:.4f\}$ “

 f”(Cramér-Rao for $n=100$: $\pm\{np.sqrt(1/(100*f)):.4f\}$ ”)

 print()

=== Gaussian Fisher Matrix ===

sigma_sq = 4.0

F_func = fisher_gaussian()

F_mat = F_func(sigma_sq)

F_inv = np.linalg.inv(F_mat)

print(f"=== Gaussian Fisher ($\sigma^2 = \{sigma_sq\}$) ===")

print(f"Fisher Matrix:\n{F_mat}")

print(f"Fisher Inverse:\n{F_inv}")
```

```
print()

=== Gradient descent vs natural gradient ===

gradient = np.array([0.5, 0.5])

sgd_step = -0.01 * gradient

nat_step = natural_gradient_step(gradient, F_inv, lr=0.01)

print("=== SGD vs Natural Gradient ===")

print(f"Gradient: {gradient}")

print(f"SGD step: {sgd_step}")

print(f"Natural step: {nat_step}")

print(f"Natural/SGD ratio (mu): {nat_step[0]/sgd_step[0]:.1f}x")

print(f"Natural/SGD ratio (sigma^2): {nat_step[1]/sgd_step[1]:.1f}x")

print()

=== Adam approximates Fisher diagonal ===

data = np.random.normal(3.0, np.sqrt(sigma_sq), size=1000)

grad_mu = (data - 3.0) / sigma_sq
```

```
grad_sigma = -1/(2*sigma_sq) + (data - 3.0)**2 / (2*sigma_sq**2)
```

```
print("=== Adam $\approx$ Fisher Diagonal ===")
```

```
print(f"Fisher $F[\mu, \mu] = \{1/\text{sigma_sq:.4f}\}")$
```

```
print(f"Adam $E[g_\mu^2] = \{\text{np.mean(grad_mu**2):.4f}\}")$
```

```
print(f"Fisher $F[\sigma^2, \sigma^2] = \{1/(2*\text{sigma_sq**2}):.5f}\}")$
```

```
print(f"Adam $E[g_\sigma^{22}] = \{\text{np.mean(grad_sigma**2):.5f}\}")$
```

```
...
```

Output:

```
...
```

```
=== Bernoulli Fisher Information ===
```

```
 $\theta = 0.1: F(\theta) = 11.1111$ (Cramér-Rao for $n=100: \pm 0.0300$)
```

```
 $\theta = 0.3: F(\theta) = 4.7619$ (Cramér-Rao for $n=100: \pm 0.0458$)
```

```
 $\theta = 0.5: F(\theta) = 4.0000$ (Cramér-Rao for $n=100: \pm 0.0500$)
```

```
 $\theta = 0.7: F(\theta) = 4.7619$ (Cramér-Rao for $n=100: \pm 0.0458$)
```

```
 $\theta = 0.9: F(\theta) = 11.1111$ (Cramér-Rao for $n=100: \pm 0.0300$)
```

=== Gaussian Fisher ( $\sigma^2 = 4$ ) ===

Fisher Matrix:

$\begin{bmatrix} 0.25 & 0. \end{bmatrix}$

$\begin{bmatrix} 0. & 0.03125 \end{bmatrix}$

Fisher Inverse:

$\begin{bmatrix} 4. & 0. \end{bmatrix}$

$\begin{bmatrix} 0. & 32. \end{bmatrix}$

=== SGD vs Natural Gradient ===

Gradient:  $[0.5 \ 0.5]$

SGD step:  $[-0.005 \ -0.005]$

Natural step:  $[-0.02 \ -0.16]$

Natural/SGD ratio ( $\mu$ ): 4.0x

Natural/SGD ratio ( $\sigma^2$ ): 32.0x

=== Adam  $\approx$  Fisher Diagonal ===

Fisher  $F[\mu, \mu] = 0.2500$

$$\text{Adam } E[g_{-\mu^2}] = 0.2490$$

$$\text{Fisher } F[\sigma^2, \sigma^2] = 0.03125$$

$$\text{Adam } E[g_{-\sigma^{22}}] = 0.03104$$

...

— -

## ## The Sentence That Stays With You

Adam does not know about the Fisher Information Matrix.

It does not compute it. It does not name it. It does not cite the paper.

But every time it divides a gradient by the square root of a running average of squared gradients, it is doing the same thing Fisher described in 1925 — measuring how much the data tells you about each parameter, and stepping inversely.

Your optimiser already knows this matrix. You just did not know what it was called.

Now you do.

— -

## \*\*References\*\*

Fisher, R.A. (1925). Theory of Statistical Estimation. *\*Proceedings of the Cambridge Philosophical Society\**, 22(5), 700–725.

[<https://doi.org/10.1017/S0305004100009580>]

(<https://doi.org/10.1017/S0305004100009580>)

Amari, S. (1998). Natural Gradient Works Efficiently in Learning. *\*Neural Computation\**, 10(2), 251–276. [<https://doi.org/10.1162/089976698300017746>]

(<https://doi.org/10.1162/089976698300017746>)

Kingma, D.P. and Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR 2015. arXiv:1412.6980. [<https://arxiv.org/abs/1412.6980>]

(<https://arxiv.org/abs/1412.6980>)

Martens, J. and Grosse, R. (2015). Optimizing Neural Networks with Kronecker-Factored Approximate Curvature. ICML 2015. arXiv:1503.05671. [<https://arxiv.org/abs/1503.05671>](<https://arxiv.org/abs/1503.05671>)

Model +

Matrix +

Machine Learning +

Data Science ✓

Artificial Intelligence +



**Written by Dr Swarneendu AI**

1.8K followers · 1K following

Following ▾

AI Learner and Ph.D.(in not so famous domain). I demystify AI using First Principles, Causal ML, and the beauty of classical math. Let's make machines reason.



